

PodiumD Hosting Standards - 2025Q4

Version	Date	Author	Comments by	Comments
0.1	20/02/2024	Jim Leitch	Sander vd B, Andrew M	
0.8	4/3/2024	Jim Leitch	Stephan Z, Jesse H, Mahmut C, Petra C	
1.0	22/4/2024	Jim Leitch	SSC/Maykin/Dimpact	
1.1	6/5/2024	Jim Leitch		
1.2	29/5/2024	Jim Leitch	Dimpact/ICATT	Updated "k8s operators"
2024Q3	27/8/2024	Jim Leitch	SSC	Converted to Markdown
2025Q4	17/12/2024	Jim Leitch	SSC, ICATT, DIMPACT, CAPGEMENI	

Executive Summary

The PodiumD hosting environment runs a collection of Common Ground-Compliant resources currently hosted on Microsoft Azure.

Development, Test, Acceptance and Production (OTAP) environments are provided for multiple Dutch city councils (municipalities) to host the PodiumD application, a replacement for the current E-suite workflow system as used by many municipalities.

PodiumD is a collection of Open-Source components built by external software developers funded by the municipalities.

This document is meant to formalize what Dimpact (the contracting party) expects from software publishers to be able to publish their software to the PodiumD platform that can be reliably deployed and operated on the PodiumD platform hosted by SSC Hosting.

The document will be hosted in a GIT repository and will be updated with versioning by means of Git Pull Requests.

Document Conventions

We will use RFC-style terms to describe the level of requirements. As time goes on various aspects of this contract will be made more and less forceful: **"MUST"/"MUST NOT"/"SHOULD"/"SHOULD NOT"/"MAY"**

([RFC 2119: Key words for use in RFCs to Indicate Requirement Levels \(rfc-editor.org\)](#))

Multiple Environments Require Easy Deployments

It should be clear that applications being deployed will be deployed for multiple municipalities in multiple OTAP environments. This means that 100% automated deployment is a hard requirement for any applications being deployed.

Hot Items list

While this document is a work in progress, the following items are high priority and should be addressed . These items are the minimum kubernetes stability requirements of the software delivered by the development partners.

We have reviewed the various components of the development partners in to determine whether they are compliant or non-compliant. If not compliant, please include these in the adjustments to your software. The goal is to ensure that all software complies with these "hot items" by the end of Q1 2026.

Health Checks and Probes

Clear health checks and probes via standard API endpoints. See section [COMP014-healthchecks], especially [COMP014.1-health-states] and [COMP014.2-probe-config] for more details.

Upgradability

Applications must be able to be upgraded in a zero-downtime manner. Use database techniques that allow for a rolling upgrade of components, migration of the database and rollback of the database when required. See section [COMP006.2-upgrade-time-limit] for more details.

High Availability

Critical components must be able to survive a zone failure. See section [COMP014.3-redundancy] for more details. Suppliers should also already be at least thinking about deploying to multiple regions and the database implications thereof.

Infrastructure Description

Subscriptions / Environments

Each customer/gemeente utilises a set of Azure Subscriptions. One subscription for each of the OTAP environments. This simplifies billing and provides security demarcation between the various environments, at customers, suppliers and OTAP level.

Not every customer/supplier requires the full set of OTAP environments.

Cloud Resource Provider

Dimapact currently uses Microsoft's Azure as our cloud platform. Our focus is currently on Microsoft Azure; however, we MAY in the future to use alternative cloud platforms. Any efforts made for deployment to our cloud platform should be generic, using open standards, keeping in mind that we may move to an alternative or even multi-cloud platform in the future.

Azure Resources

The resources currently in our Azure cloud environment, in use and available are described below. In time we may migrate or simultaneously deploy in an alternative cloud environment such as Google / AWS / Private cloud.

Any new Azure resources required can be discussed going forward, this is current list:

- **Kubernetes** (k8s) – AKS
 - We use standard AKS with two autoscaling groups of nodes, one for all application workloads, another for all management-type workloads.
- **Container Registry** – ACR
 - ACR with vulnerability scanning
- **SQL DBMS** - Azure Database for PostgreSQL flexible servers
 - PostgreSQL 14 is our standard database server, one instance per environment with multiple databases inside the server, one per application.
 - Database Timezone is UTC
- **Edge Load Balancer** - Azure Application Gateway
 - Azure Load Balancer is our internet-facing endpoint, providing SSL termination, HTTP -> HTTPS redirection, URL rewriting and distribution of traffic to the appropriate K8S cluster. We have one load balancer for ONTW/TEST and another for ACCP/PROD.
 - We may in future choose to use our on-premises Palo Alto and GW services for this function
- **Storage** - Azure Storage Accounts
 - Azure storage accounts are used for provision of Blob storage and Azure File Shares (SMB and NFS)
- **Secret and Certificate Management** – Azure Key Vault
 - Azure Key Vault is the single source of truth for all certificates and secrets
- **Identity Management** - Active Directory / Entra ID

- Azure Entra ID SHOULD be used as the only method of Identity Management. If another identity tool is used, a waiver must be obtained with a plan for future migration to Entra.
- **Deployment Orchestration** - Azure DevOps
 - Azure DevOps is our CI/CD orchestration tool, used for deployment of infrastructure, application deployment and other management tasks. We use a mixture of Terraform and bash scripting with az cli commands to build and maintain our infrastructure.
- **Infra Lifecycle**
 - Most Azure infrastructure resources (storage, databases) have very long lifecycles.
 - Kubernetes versions are handled frequently with the assistance of Azure Fleet Manager and will be updated on a regular monthly cycle **ontw/test/accp/prod** with one week between each environment type, meaning that any non-critical k8s patches will have been running in ONTW for 3 weeks before production is patched
 - Upgrades to other resources such as RDBMS and storage will be announced well in advance to allow suppliers to test their applications with the new versions and to work on and plan an upgrade path for the application if required.

Azure Application Gateway

Azure application gateway can rewrite HTTP headers. At this moment we apply only one header rewrite for HSTS purposes as follows:

We have not made any other changes to the AGW settings. For any other default settings please refer to the MS Azure documentation, any changes to default settings will be reflected in this document.

HTTP2 support is currently DISABLED in the AGW.

Non – Azure Native Shared Resources

Some resources are not available as Azure native services and are instead deployed inside the Kubernetes cluster:

SSL Certificates (FrontEnd)

- We use SSL certificates from **LetsEncrypt**. The certificates are automatically requested and downloaded for every web facing service by the cert-manager application inside the cluster. Certificates and keys are then stored in the Azure Key Vault. They are then subsequently deployed to the listeners in the Application Gateway.
- SSL Certificates used internally between apps and legacy system are

out-of-scope of this discussion.

Cost management

In future we may be able to provide extra cost management tooling using the billing data stored.

Environments (OTAP)

Each municipality and organization has the possibility to run O,T,A and/or P organizations, defined as follows:

DEVELOPMENT(ONTWIKKEL)

Ontwikkel (development) environments are meant to verify proper working of the software in a production-similar environment. Developers are free to deploy container versions and new helm charts in this environment. Changes to Azure resources and the k8s cluster are performed by SCC hosting in an updated version of the environment.

To save on cloud costs we switch all dev, test and acceptance environment off in the evening and at weekends. If any individual environments need to be kept on for any reason for a short period of time, this can be accommodated.

Test

Test environments are meant for the municipalities to test new application features and provide training facilities to staff.

Acceptance

Acceptance environments are 100% like-for-like with production environments. Hardware resources are the same as production and copies of production data.

Production

Production is of course the environment per customer/municipality that has maximum uptime, monitoring, and resources.

Deployment- Azure DevOps Pipelines

Deployment of applications to the k8s cluster are done by an Azure DevOps Pipeline to deploy a Helm Chart. Any other infrastructure preparation is done by Terraform files and/or bash scripts running "az cli" commands.

What We Expect

We expect that Dimpact will require suppliers to deliver the applications in a way that allows for secure, scalable, stable, manageable, and easily deployable systems.

Standards Summary Table

Standard ID	Summary
CORE REQUIREMENTS	Basic application structure, deployment, and configuration
COMP001-naming	Naming conventions for services, components, and resources
COMP002-versioning	Versioning requirements for PodiumD components
COMP002.1-version-api	Standardized version API endpoint exposing application metadata
COMP003.1-app-as-containers	Applications must be delivered as containers
COMP003.3-container-registry	Containers must be available via public container registry
COMP003.3-semantic-versioning	Container labels must use semantic versioning
COMP004-timezone	Applications must use UTC timezone for logging and monitoring
COMP005-software-supply-chain	Container vulnerability scanning and mitigation requirements
COMP006.1-stop-start-safe	Applications must withstand sudden stops and starts
COMP006.2-upgrade-time-limit	Zero-downtime rolling upgrades with rollback capability
COMP006.3-rollbacks	Component rollback requirements and procedures
COMP007.1-helm	All applications must be accompanied by Helm charts
COMP007.3-dependancy-apps	Dependency applications must be deployed as Helm sub-charts
COMP007.4-storage	Helm chart must support SSC's storage choices
COMP007.5-ingress	Helm chart must support SSC's ingress choices
COMP007.5-database	Helm chart must support external cloud database usage
COMP008.1-100percent-automated	Complete deployment automation from Helm chart
COMP009-internal-communication	Inter-component communication should occur inside cluster via APIs
COMP010-sql-migration-scripts	Database migrations must be built-in with rollback functionality
COMP011-no-local-storage	Persistent data must not be written to local container storage

COMP011-permanent-data	Permanent data requiring backup must be written to external file share
COMP012-secrets	Secret information must be written to K8s secrets store
COMP013-ssl	Application endpoints must be presented as HTTP (SSL terminated at gateway)
HEALTH CHECKS & MONITORING	Application observability, health, and metrics
COMP014-healthchecks	Applications must provide probe endpoints for container health monitoring
COMP014.1-health-states	Clear definition of health states (healthy, degraded, unhealthy, starting, stopping)
COMP014.2-probe-config	Implementation of liveness, readiness, and startup probes
COMP014.3-redundancy	Multi-zone deployment with retry mechanisms and load balancing
COMP014.4-pdb	Critical services must define Pod Disruption Budgets
COMP015.1-logging-to-stdout	Applications must stream all logging to STDOUT
COMP016.2-utc-logs	Log lines must use UTC timestamps without timezone
COMP017-golden-signals	Applications must provide metrics for the four golden signals
COMP017.1-latency-metrics	Request duration and response time metrics
COMP017.2-traffic-metrics	Requests per second and concurrent connections metrics
COMP017.3-error-metrics	Error rate and error types metrics
COMP017.4-saturation-metrics	Resource utilization and queue depth metrics
COMP018-alerting	Alert design principles based on Google SRE practices
COMP018.1-slo-alerting	Alerts should be tied to Service Level Objectives
COMP019-slo	Service Level Objectives and error budget definitions
OPERATIONS & DEPLOYMENT	Resource management, recovery, and release practices
COMP020-resource-recommendations	Applications must provide CPU/memory estimates and autoscaling configuration
COMP021-disaster-recovery	Recovery Time Objective (RTO) and Recovery Point Objective (RPO) requirements
COMP022-release-notes	Comprehensive release notes with breaking changes and migration steps

SECURITY, COMPLIANCE & PERFORMANCE	Security controls, compliance, and optimization
COMP023-security	Container security, network security, and secret management requirements
COMP024-sbom	Software Bill of Materials in SPDX or CycloneDX format
COMP025-api-practices	RESTful API design standards, versioning, and documentation
COMP028-compliance	Audit logging and GDPR compliance requirements
COMP029-performance	Response time targets, throughput requirements, and caching strategy
COMP030-cost-optimization	Resource efficiency and cost visibility requirements

Naming Conventions [COMP001-naming]

All application and parameter naming should be uniform to allow for a minimum of repeated configuration.

Service Naming Standards:

- Service names MUST follow a consistent pattern:
`{domain}-{function}-service` (e.g., `zaak-processing-service`, `klant-api-service`)
- Service names MUST use lowercase with hyphens as separators
- Service names MUST be descriptive and reflect the business domain
- Service names MUST be consistent across all environments (OTAP)
- Helm chart names MUST match the service name
- ~~Kubernetes namespaces SHOULD follow the pattern: `{customer}-{service-name}` or use a single namespace per recognizable component~~
- Container image names MUST use the service name as base
- Applications MUST have one single name that is used throughout the service, preferable between 5 and 10 characters
- API endpoint paths SHOULD use the service domain name for consistency

Naming Examples:

- Good: `klant-management-service`, `zaak-api-service`, `document-storage-service`
- Avoid: `service1`, `api`, `app-x`, `temp-service`

Dimpart will provide examples of parameter naming to be used in the umbrella helm chart.

Versioning [COMP002-versioning]

PodiumD consists of a suite of sub-applications. The PodiumD version is defined by the versions of the sub-applications.

Version API Endpoint [COMP002.1-version-api]

- [COMP002.1-version-api] Each application/component MUST expose a standardized version API endpoint at `/api/v1/version` or `/health/version`
- The version endpoint SHOULD return the following information in JSON format:
 - Application/component name
 - Semantic version number
 - Build date/timestamp
 - Git commit SHA (optional but recommended)
 - Dependencies and their versions (optional but recommended)
- Example response:

```
1  {
2    "name": "zaak-api-service",
3    "version": "2.5.1",
4    "buildDate": "2024-10-15T14:30:00Z",
5    "gitCommit": "a3f2d1c",
6    "dependencies": {
7      "postgres": "14.5",
8      "redis": "7.0.5"
9    }
10 }
```

- The version endpoint MUST be accessible without authentication
- The version endpoint SHOULD be included in health check monitoring

Containers

- [COMP003.1-app-as-containers] Applications MUST be delivered as one or a set of application containers
- [COMP003.3-container-registry] These containers MUST be available via a public Container Registry such as **Docker Hub**
- Non-open-source containers MAY be available secured by a key/token.
- [COMP003.3-semantic-versioning] Container labels MUST be "semantically versioned."
- Pre-release versions MAY be deployed with version + keyword (e.g., 1.2.3-snapshot)

SSC hosting will copy containers from the supplier's own container registry to a Container Registry in the SSC Azure environment.

Timezone [COMP004-timezone]

For logging and monitoring purposes, the timezone for applications should be **UTC**. The default timezone for the database is UTC, all logging and metrics should be set to provide UTC output. Of course any end-user interaction should be presented to the client in the appropriate local timezone.

Software Supply Chain [COMP005-software-supply-chain]

SSC Hosting will scan all containers for vulnerabilities on a regular basis. Containers delivered **MUST** be already scanned for vulnerabilities by the supplier. Any vulnerabilities that are flagged **MUST** be fixed, mitigated and/or receive a waiver from all system architects of supplier + Dimpact + SSC Hosting within a specified time limit.

An example of code-level scanning would be SonarQube:

<https://www.sonarsource.com/products/sonarqube/>

An example of container scanning would be GitHub container scanning:

<https://github.com/marketplace/actions/container-scan>

Scalability and Startup/Shutdown

Apart from where inappropriate, applications **SHOULD** gracefully handle load-based up and down-scaling of containers.

[COMP006.1-stop-start-safe] Application **MUST** be able to withstand sudden stops and starts (for example a node failing or container-node migration event)

This means that to be able to withstand a node failure, and to ensure no downtime, all components **MUST** have a minimum of two instances (that will automatically take the work over from the other in event of failure) and make use of shared storage.

Applications **MAY** be deployed with init containers, startup and shutdown scripts.

Component Upgrade Time Limits [COMP006.2-upgrade-time-limit]

- [COMP006.2-upgrade-time-limit] Any upgrade of a component **SHOULD** be able to be performed in a rolling manner, with zero downtime
- Deployments that require any downtime require architectural review
- Deployments should provide rollback function, this is especially needed in the case of migrations

that are stop due to errors during deployment (see also COMP010-sql-migration-scripts)

Component Upgrade Time Limits [COMP006.3-rollbacks]

Helm Charts Deployment

- Dimpact will curate and provide all required Helm charts in one single repository
- [COMP007.1-helm] All applications MUST be accompanied by Helm Charts. Applications MAY use helm sub-charts and MUST be able to be deployed as a sub-chart themselves to facilitate and contribute to a complete "umbrella" installation of the PodiumD platform.
- ~~[COMP007.2-operators] We do not support the use of applications and components deployed by means of Kubernetes Operators unless the installation and use of the operators can be encapsulated inside a single Umbrella Helm chart.~~
- [COMP007.3-dependancy-apps] Any dependency-applications MUST be deployed as Helm sub-charts.
- The application itself MUST be deployable as a sub chart as part of an over-encompassing PodiumD Helm chart. Over time all parties should work to harmonize variable naming in the Helm charts
- Deployments SHOULD be performed as a rolling update – unless waiver has been granted. Deployments and upgrades that involve database changes are currently exempt from this requirement.
- Helm charts SHOULD be available via the same artifacts' repository as the containers and SHOULD be made available in OCI format.
- [COMP007.4-storage] Helm chart MUST support storage type via SSC's own storage choices (if file storage is required)
- [COMP007.5-ingress] Helm chart MUST support ingress that allows for SSC's own ingress choices (if ingress in required)
- [COMP007.5-database] Helm chart MUST support external cloud database usage (if database is required)

K8s Namespaces

PodiumD as a whole is deployed in one single "podiumd" kubernetes namespace.

~~SSC hosting uses one single k8s namespace per recognizable (openforms, objects etc.) component. Components must be deployed into its own namespace in Kubernetes cluster.~~

~~The name of the namespace will always be the same as the name of the application as defined in the helm chart.~~

100% Automation

[COMP008.1-100percent-automated] PodiumD software will be deployed to many customers and environments, for this reason, all components of the application MUST be possible to deploy completely from the Helm chart.

In this case – 100% deployment means the containers running correctly inside the K8s cluster with the correct deployment parameters and secrets, connecting to other resources such as databases and other PodiumD applications.

Final configuration "inside" of the application is out of this scope.

To help understanding of each area of deployment, development party Maykin Media produced this useful set of definitions:

- **OTAP-omgevingen**

Ontwikkel-, Test-, Acceptatie, Productie-omgevingen

- **Platform**

Set aan componenten die in samenhang een oplossing bieden (bijv. Open Zaak en ZAC)

- **Installatie**

Het deployen van een component (bijv. Helm chart uitvoeren zodat Open Zaak en ZAC technisch draaien)

- **Configuratie**

Parameters zodat de component kan doen wat het moet doen (bijv. de koppeling tussen ZAC en Open Zaak)

- **Inrichting**

Functionele vulling van het component evt. met hele specifieke configuratie (bijv. zaaktypen, processen)

Application Addressing

Dim pact will provide application URLs for all public facing applications.

Dim pact will liase with gemeentes as to the application addressing used and for crestion of appropriate hostnames and cnames.

Communication Between Components

[COMP009-internal-communication] Communication between components inside the same k8s cluster SHOULD take place inside the cluster. Communication and data transfer between components SHOULD take place via the APIs.

Database Migrations

[COMP010-sql-migration-scripts] Applications MUST have all database migrations built in or provide config scripts for a DB migration tool such as Flyway or Liquibase, or that as provided by the framework (e.g. Django)

Database migrations MUST provide a rollback function

Application concurrency / Statelessness

- Applications SHOULD be stateless and commit all data to a database.
- Files SHOULD be written to an object store such as Azure Blob Storage
- Files MAY be written to cloud file shares.
- [COMP011-no-local-storage] Persistent files containing any data MUST NOT be written to local storage on the container but instead to a kubernetes PVC.
- [COMP011-permanent-data] Permanent data that requires to be backed up must be written to a file share created outside of the K8s cluster

FILES

Files MAY be stored in the following storage types, in order of preference:

- Object Storage
- SMB file share shared volume file storage
- Persistent volume storage

Configuration / Secrets

- All application configurations SHOULD be performed via environment variables.
- Environment variables SHOULD be set via config maps in the helm chart.
- [COMP012-secrets] Secret/sensitive information MUST be written to the K8s secrets store
- Helm charts SHOULD provide the ability to source secrets from the secrets store directly - Commonly this is called the secretKeyRef (externalized secrets) pattern - templates use refs (env.valueFrom.secretKeyRef) instead of embedding secret values in values.yaml.
- Secrets will be sourced and read into the secrets store from the Azure Key Vault at deployment-time.
- Application logging verbosity SHOULD be enabled by setting a debug environment variable and restarting the container.

SSL Certificates

[COMP013-ssl] Application end points MUST be presented as HTTP, unencrypted. Our Public-facing Azure Application Gateway performs SSL termination and guarantees a secure path to the k8s cluster via the Azure VWAN and Virtual Hub facilities. The Azure Application Gateway listeners are populated by Lets Encrypt SSL certificates at deploy-time

Health Checks and Probes

[COMP014-healthchecks] Applications MUST ensure availability of probe endpoints to allow for correct handling of starting, running and failing containers.

Enhanced Health Check Requirements based on Azure Well-Architected Framework:

Health State Definitions [COMP014.1-health-states]

A solid Kubernetes setup uses distinct startup, liveness, and readiness probes: startup ensures full initialization (migrations, warm A robust Kubernetes setup employs distinct startup, liveness and readiness probes. Startup ensures complete initialisation, including migrations and warm caches. Liveness is a low-cost internal heartbeat process (no external calls) and readiness confirms dependencies like a simple database ping, cache/broker connectivity and config/secrets loading before routing traffic.

Broader readiness health should also validate resource pressure, such as CPU throttling, memory nearing limits, disk space/inodes thresholds and queue/backlog size against SLAs. Thread/worker pool saturation, recent error rate spikes and optionally feature flags or external auth token freshness should also be monitored while maintaining fast response times (sub-100ms).

Complement these probes with a /metrics endpoint that exposes latency histograms, error counters, request concurrency, queue depth, GC pauses and custom saturation gauges. This enables autoscaling (HPA/KEDA) and alerts on trends rather than simply binary probe failures.

Applications MUST clearly define and document the following operational states:

1. **Healthy/Normal:** System operates within expected parameters
 - All dependencies available
 - Response times within SLA
 - Error rates below threshold
 - Resources within normal limits
2. **Degraded:** Reduced functionality but still operational
 - Some non-critical dependencies unavailable
 - Response times elevated but acceptable
 - Operating in fallback mode

- Should trigger warnings but not critical alerts
- 3. **Unhealthy/Failed:** System not functioning correctly
 - Critical dependencies unavailable
 - Unable to process requests
 - Data consistency issues
 - Should trigger immediate alerts
- 4. **Starting:** System is initializing
 - Loading configuration
 - Establishing database connections
 - Warming up caches
 - Not ready for traffic
- 5. **Stopping:** Graceful shutdown in progress
 - Completing in-flight requests
 - Closing connections
 - Cleaning up resources

Kubernetes Probe Configuration [COMP014.2-probe-config]

Applications MUST implement all three types of probes:

Liveness Probe:

- Determines if container needs to be restarted
- MUST be simple and fast (< 1 second response time)
- Should NOT check dependencies
- Example: Simple HTTP endpoint returning 200 OK if application can accept requests

Readiness Probe:

- Determines if container can receive traffic
- SHOULD check critical dependencies (database, required APIs)
- MAY have longer timeout than liveness probe
- Should transition pod out of service gracefully

Startup Probe:

- For slow-starting applications
- Disables liveness/readiness checks during startup
- Prevents premature container restarts
- Should have generous initial delay and timeout

Probe Best Practices:

- Initial delay should account for realistic startup time
- Failure threshold should avoid false positives from temporary issues
- Success threshold should prevent flapping
- Period should balance responsiveness with load
- Timeout should account for network variability

See an example of a section of a helm chart that defines the health probes for a component:

```

1  startupProbe:
2    initialDelaySeconds: 40
3    periodSeconds: 10
4    timeoutSeconds: 1
5    failureThreshold: 12
6    successThreshold: 1
7
8  livenessProbe:
9    initialDelaySeconds: 0
10   periodSeconds: 10
11   timeoutSeconds: 1
12   failureThreshold: 12
13   successThreshold: 1
14
15  readinessProbe:
16    initialDelaySeconds: 0
17    periodSeconds: 5
18    timeoutSeconds: 1
19    failureThreshold: 3
20    successThreshold: 1
21
22  probesEnabled:
23    startupProbe: true
24    livenessProbe: true
25    readinessProbe: true

```

See [Configure Liveness, Readiness and Startup Probes | Kubernetes](#)

Redundancy and Fault Tolerance [COMP014.3-redundancy]

Based on Azure Well-Architected Framework reliability pillar:

Multi-Zone Deployment:

- Production deployments SHOULD spread replicas across multiple Azure Availability Zones
- Minimum 3 replicas for critical services
- Pod anti-affinity rules SHOULD be used to prevent co-location

Retry Mechanisms:

- Applications **MUST** implement exponential backoff for retrying failed requests
- Maximum retry attempts should be configurable
- Circuit breaker pattern **SHOULD** be implemented for external dependencies

Load Balancing:

- Traffic **MUST** be distributed across multiple instances
- Session affinity **SHOULD** be avoided where possible for better distribution
- Health checks must remove unhealthy instances from rotation

Pod Disruption Budgets:

- [COMP014.4-pdb] Critical services **MUST** define Pod Disruption Budgets
- Minimum available pods should ensure service continuity during updates
- Should balance update velocity with availability requirements

Logging / Monitoring / Metrics / Alerting

Monitoring

Monitoring is the combination of aggregating logs and metrics to ascertain if the application is working as expected, hopefully before the end user is affected and allow us to react in time to fix the problem and/or underlying issue.

Infra monitoring

The infrastructure (resources such as k8s, edge gateway, databases etc.) **SHALL** be monitored for errors as exported to Azure. Other metrics such as CPU capacity, disk IO should be used to ensure rightsizing and autoscaling of resources.

Logging

- [COMP015.1-logging-to-stdout] Applications **MUST** stream all logging to STDOUT. These logs will be captured and sent to the appropriate logging endpoint.
- Application logging **MUST** be configurable via a LOGLEVEL environment variable following standard usage for DEBUG, INFO, WARN, ERROR, and FATAL log levels.
- Applications logs **SHOULD** be written in JSON format to allow proper parsing in the Logging system. (see <https://betterstack.com/community/guides/logging/json-logging/>)

- Applications MAY write relevant logging information in a form that can be read in the application, but this MUST be in addition to the logs written to STDOUT.
- [COMP016.2-utc-logs] Log lines MUST be written with timestamps in UTC format, no time zone.

Metrics - Golden Signals [COMP017-golden-signals]

Based on Google SRE best practices, applications MUST provide metrics for the four golden signals:

1. Latency [COMP017.1-latency-metrics]

- Request duration/response time
- Separate success and error latencies
- Percentile distributions (p50, p95, p99)
- Example metrics:
 - `http_request_duration_seconds`
 - `api_response_time_milliseconds`

2. Traffic [COMP017.2-traffic-metrics]

- Requests per second
- Concurrent connections
- Active users
- Example metrics:
 - `http_requests_total`
 - `active_connections`
 - `concurrent_users`

3. Errors [COMP017.3-error-metrics]

- Error rate (errors/total requests)
- Error types and codes
- Failed transactions
- Example metrics:
 - `http_requests_failed_total`
 - `transaction_errors_total`
 - `error_rate_percentage`

4. Saturation [COMP017.4-saturation-metrics]

- Resource utilization (CPU, memory, disk, network)
- Queue depth

- Thread pool usage
- Database connection pool usage
- Example metrics:
 - `cpu_usage_percentage`
 - `memory_usage_bytes`
 - `queue_depth`
 - `db_connection_pool_active`

Additional Application Metrics:

Applications SHOULD provide metrics that make sense for the application in hand, for example:

- Number of Transactions
- Transaction latency
- Incomplete transactions
- Failed transactions
- Any other relevant metrics per application

Metrics Implementation:

- Metrics SHOULD be exposed in Prometheus format at `/metrics` endpoint
- Metrics MUST include appropriate labels for filtering (environment, version, instance)
- Metrics collection SHOULD have minimal performance impact (< 1% overhead)
- Counter metrics SHOULD be monotonically increasing
- Gauge metrics SHOULD represent current values
- Histogram metrics SHOULD be used for latency measurements

The metrics can then be scraped by (for example) Prometheus to display in graph form, for analysis and for alerting purposes.

Metrics are an ongoing discussion and will always be updated and improved throughout the lifecycle of the application.

The following article gives a thorough discussion on the use of metrics usage in Kubernetes:

<https://sysdig.com/blog/golden-signals-kubernetes/>

Alerting [COMP018-alerting]

Based on Google SRE principles:

Alert Design Principles:

- Alerts MUST only notify when human intervention is required

- Alerts SHOULD be based on symptoms (user impact) not causes
- Every alert MUST be actionable
- Alert descriptions MUST include:
 - What is broken
 - Impact on users/business
 - Next steps for investigation
 - Relevant runbook links

Alert Severity Levels:

1. **Critical/P1:** Production down, major functionality broken, immediate action required
2. **Warning/P2:** Degraded performance, potential issue, action required within business hours
3. **Info/P3:** Informational, no immediate action required

SLO-Based Alerting [COMP018.1-slo-alerting]:

- Alerts SHOULD be tied to Service Level Objectives (SLOs)
- Use error budget burn rate for alerting thresholds
- Fast burn alerts (1-2 hour windows) for immediate issues
- Slow burn alerts (24-72 hour windows) for trend issues

Alert Threshold Guidelines:

- Error rate: Alert when > 1% of requests fail (or based on SLO)
- Latency: Alert when p95 > SLO threshold
- Saturation: Alert when > 80% resource utilization
- Availability: Alert when uptime < SLO (e.g., 99.9%)

Alert Fatigue Prevention:

- Avoid alerting on predicted problems
- Use multi-window, multi-burn-rate alerting
- Regularly review and tune alert thresholds
- Remove alerts that don't result in action

Service Level Objectives (SLO) [COMP019-slo]

SLO Definition Requirements:

Each production service MUST define:

1. **Service Level Indicators (SLIs):**
 - Availability: Percentage of successful requests
 - Latency: Request response time (p95, p99)

- Throughput: Requests processed per second
- Error rate: Percentage of failed requests

2. **Service Level Objectives (SLOs):**

- Target values for each SLI
- Measurement window (e.g., 30 days)
- Example SLOs:
 - Availability: 99.9% (43.2 minutes downtime/month)
 - Latency: 95% of requests < 200ms
 - Error rate: < 0.1% of requests

3. **Error Budgets:**

- Calculated from SLO (e.g., 0.1% = error budget)
- Used to balance velocity vs. reliability
- When budget exhausted, freeze feature releases and focus on reliability

SLO Documentation:

- SLOs **MUST** be documented in service documentation
- SLOs **SHOULD** be reviewed quarterly
- SLO violations **MUST** trigger post-incident reviews

Resource requirements

[COMP020-resource-recommendations] As part of the Helm chart, applications **MUST** provide an estimate of CPU/Disk/Memory for a certain baseline, with resources required to operate the application

Resource Specification Requirements:

Applications **MUST** specify:

Requests (Guaranteed Resources):

- Minimum CPU (in millicores)
- Minimum Memory (in MB/GB)
- These represent guaranteed allocation

Limits (Maximum Resources):

- Maximum CPU (in millicores)
- Maximum Memory (in MB/GB)
- Prevents resource hogging

For Example:

For application **X** to be able to process **Y** transactions per minute, we recommend:

- Requests: 1000 milliCPU, 512MB RAM
- Limits: 2000 milliCPU, 1GB RAM
- 3 container replicas (minimum for high availability)
- File storage type and size requirements
- Network bandwidth requirements (if applicable)

Autoscaling Configuration:

- Horizontal Pod Autoscaler (HPA) thresholds
- Target CPU utilization (typically 70-80%)
- Minimum and maximum replica counts
- Scale-up and scale-down behavior

Disaster Recovery [COMP021-disaster-recovery]

Based on Azure Well-Architected Framework:

Recovery Objectives:

Each application MUST define:

1. **Recovery Time Objective (RTO):**
 - Maximum acceptable downtime
 - Time to restore service after incident
 - Should align with business requirements
2. **Recovery Point Objective (RPO):**
 - Maximum acceptable data loss
 - Frequency of backups/replication
 - Should align with business requirements

Backup and Restore:

- Database backup strategy MUST be documented
- Backup frequency MUST meet RPO requirements
- Restore procedures MUST be tested quarterly
- Backup retention policy MUST be defined

Disaster Recovery Testing:

- DR procedures MUST be tested at least annually
- Chaos engineering tests SHOULD be performed regularly
- Failover and failback procedures MUST be documented
- DR test results MUST be documented

DNS Naming Conventions

Applications MUST support access from ANY URL simultaneously. There should be no restriction on changing application URLs at any point in time. If a gemeente decide to change the URL that citizens access the application from, this should not require any database updates.

~~Applications MUST be accessible via:~~

- ~~• technical domains: app.test.gemeente.dimpact.nl~~
- ~~• vanity name domains: burgerformulieren.gemeente.nl~~
- ~~• kubernetes internal DNS naming: openzaak.podiumd.svc.cluster.local~~
- ~~• relative path domain name: gemeente.nl/formulieren~~

More information can be found here:

<https://dimact.atlassian.net/wiki/spaces/PCP/pages/175865864/Toegang+API+s+via+verschillende+domeinnamen+waaronder+interne+toegang>

~~Deployment Verzoek~~

~~Deployment requests take the form of a "Deployment Verzoek", a form filled in with all relevant data required for deployment.~~

Release notes

[COMP022-release-notes] The following information MUST be contained in the application release notes:

- Full description of parameters needed for HELM deploy.
- Full description of possible output parameters (eg API keys generated at deploy time)
- Dependencies on earlier versions of the application
- Storage Requirements
- Database requirements
- External access / Ingress requirements
- **Breaking changes** clearly marked and explained
- **Security fixes** with CVE numbers if applicable
- **Migration steps** if required for upgrade
- **Rollback procedures** in case of issues
- **Performance impact** of changes
- **Known issues** in this release

Release notes SHOULD follow the "Keep a Changelog" format:

- <https://keepachangelog.com/>

Security Requirements [COMP023-security]

Container Security

- Containers MUST run as non-root user
- Containers MUST use read-only root filesystem where possible
- Containers MUST drop all unnecessary Linux capabilities
- Container images MUST be scanned for vulnerabilities before deployment
- High/Critical CVEs MUST be addressed within 7 days

Network Security

- Network policies MUST be defined to restrict pod-to-pod communication
- Only necessary ports SHOULD be exposed
- All external communication MUST use TLS 1.2 or higher
- API authentication and authorization MUST be implemented

Secret Management

- Secrets MUST NEVER be committed to source control
- Secrets MUST be stored in Azure Key Vault
- Secrets SHOULD be rotated regularly (at least annually)
- Application SHOULD support secret rotation without restart where possible

SBOM (Software Bill of Materials) [COMP024-sbom]

- Applications SHOULD provide SBOM in SPDX or CycloneDX format
- SBOM SHOULD include all direct and transitive dependencies
- SBOM SHOULD be updated with each release
- SBOM helps with vulnerability tracking and compliance

Integration and API Best Practices [COMP025-api-practices]

API Design Standards

RESTful Principles:

- Use standard HTTP methods (GET, POST, PUT, PATCH, DELETE)
- Use plural nouns for collections (`/users` , `/orders`)
- Use nested resources for relationships (`/users/{id}/orders`)
- Return appropriate HTTP status codes

Versioning:

- API versions MUST be included in URL path (`/api/v1/resource`)

- Major version changes MUST be backward incompatible only
- Old API versions SHOULD be supported for at least 12 months after deprecation

Error Handling:

- Errors MUST return consistent JSON format
- Include error code, message, and details
- Use appropriate HTTP status codes

Rate Limiting:

- APIs SHOULD implement rate limiting
- Rate limit headers SHOULD be returned
- Rate limits SHOULD be documented

Documentation:

- APIs MUST provide OpenAPI/Swagger documentation
- Documentation MUST be kept up-to-date
- Example requests and responses SHOULD be provided

~~Service Mesh Considerations [COMP026-service-mesh]~~

~~For complex microservice environments:~~

- ~~Service mesh (like Istio or Linkerd) MAY be used for:~~
 - ~~Mutual TLS between services~~
 - ~~Traffic management and routing~~
 - ~~Observability and tracing~~
 - ~~Circuit breaking and retry logic~~
- ~~If service mesh is used:~~
 - ~~Applications MUST NOT implement their own service discovery~~
 - ~~Applications SHOULD rely on mesh for retry and timeout logic~~
 - ~~Applications MUST expose metrics for mesh integration~~

~~Documentation Requirements [COMP027-documentation]~~

Technical Documentation

~~Each application MUST provide:~~

1. **Architecture Documentation:**
 - ~~System architecture diagram~~
 - ~~Component interactions~~

- Data flow diagrams
- External dependencies

2. **Deployment Documentation:**

- Helm chart values explanation
- Environment specific configurations
- Deployment sequence/dependencies
- Rollback procedures

3. **Operations Documentation:**

- Runbooks for common issues
- Troubleshooting guides
- Log interpretation guide
- Metrics dashboard descriptions

4. **API Documentation:**

- OpenAPI/Swagger specification
- Authentication/authorization details
- Rate limiting information
- Example requests/responses

5. **Security Documentation:**

- Security controls implemented
- Authentication/authorization model
- Data encryption details
- Compliance requirements

Documentation Maintenance

- Documentation MUST be updated with each release
- Documentation SHOULD be version-controlled
- Documentation SHOULD be accessible to all stakeholders
- Outdated documentation MUST be clearly marked

Compliance and Audit [COMP028-compliance]

Audit Logging

- All authentication attempts MUST be logged
- All authorization failures MUST be logged
- All data access SHOULD be logged (where applicable)
- Logs MUST be retained per regulatory requirements

- Logs MUST be tamper-proof

Compliance Requirements

Applications handling personal data MUST:

- Comply with GDPR requirements
- Implement data retention policies
- Support right to erasure (right to be forgotten)
- Support data portability
- Maintain audit trails

Performance Requirements [COMP029-performance]

Response Time Targets

- Interactive requests: < 200ms (p95)
- API calls: < 500ms (p95)
- Batch operations: Document expected duration
- Database queries: < 100ms (p95)

Throughput Requirements

- Applications MUST document expected throughput
- Load testing results SHOULD be provided
- Performance degradation under load MUST be documented

Caching Strategy

- Applications SHOULD implement caching where appropriate
- Cache invalidation strategy MUST be documented
- Cache hit rate SHOULD be monitored

Cost Optimization [COMP030-cost-optimization]

Resource Efficiency

- Applications SHOULD use resources efficiently
- Unnecessary resource requests waste money
- Autoscaling SHOULD be used to match demand
- Idle resources SHOULD be scaled to minimum during off-hours

Cost Visibility

- Resource requirements SHOULD be documented

- Cost per transaction/user SHOULD be estimated
- Optimization opportunities SHOULD be identified

References

[Architecting Applications for Kubernetes | DigitalOcean](#)

[Free Guide: Our 15 Principles for Designing and Deploying Scalable Applications on Kubernetes \(elastisys.com\)](#)

<https://sysdig.com/blog/golden-signals-kubernetes/>

[Microsoft Well-Architected Framework for Azure Kubernetes Service](#)

[Google SRE: Monitoring Distributed Systems](#)

[Google SRE: Service Level Objectives](#)

[Kubernetes Best Practices](#)

[Azure Best Practices for AKS](#)